

Experiment ‘p_14_2_selection_sort_interval_seq’ Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_14_2_selection_sort_interval_seq

Natural language statement: Write a method that implements the selection sort algorithm to sort an array of values from a finite interval.

Method signature: p_14_2_selection_sort_interval_seq(inputs: seq<int>, bound: nat) returns (sorted: seq<int>)

Ensures

- forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
- multiset(inputs) == multiset(sorted)

Requires

- forall i :: 0 <= i < |inputs| ==> 0 <= inputs[i] <= bound

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that implements the selection sort algorithm to sort an array of values
↪ from a finite interval.

The signature should be:

p_14_2_selection_sort_interval_seq(inputs: seq<int>, bound: nat) returns (sorted: seq<int>
↪ >)

The method should respect the following contract:

requires forall i :: 0 <= i < |inputs| ==> 0 <= inputs[i] <= bound, ensures forall i, j
↪ :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j], ensures multiset(inputs) ==
↪ multiset(sorted)

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnKJ4thlw3PnziIeoOfr28kENhU3P

Model: gpt-5.5-2026-04-23

Created at: 1780648502

Finish reason: stop

Usage: 380 tokens in, and 9334 tokens out

```
{
  var rest := inputs;
  sorted := [];

  while |rest| > 0
    invariant multiset(sorted) + multiset(rest) == multiset(inputs)
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
    invariant forall i, j :: 0 <= i < |sorted| && 0 <= j < |rest| ==> sorted[i] <= rest[j]
    decreases |rest|
  {
    var minIdx := 0;
    var k := 1;

    while k < |rest|
      invariant 0 <= minIdx < |rest|
      invariant 1 <= k <= |rest|
      invariant forall t :: 0 <= t < k ==> rest[minIdx] <= rest[t]
```

```

    decreases |rest| - k
{
  if rest[k] < rest[minIdx] {
    var oldMinIdx := minIdx;
    minIdx := k;
    forall t | 0 <= t < k
      ensures rest[minIdx] <= rest[t]
    {
      assert rest[minIdx] == rest[k];
      assert rest[k] < rest[oldMinIdx];
      assert rest[oldMinIdx] <= rest[t];
    }
    assert rest[minIdx] <= rest[k];
  } else {
    assert rest[minIdx] <= rest[k];
  }
}

forall t | 0 <= t < k + 1
  ensures rest[minIdx] <= rest[t]
{
  if t < k {
    assert rest[minIdx] <= rest[t];
  } else {
    assert t == k;
    assert rest[minIdx] <= rest[k];
  }
}
k := k + 1;
}

assert k == |rest|;
assert forall t :: 0 <= t < |rest| ==> rest[minIdx] <= rest[t];

var oldSorted := sorted;
var oldRest := rest;
var m := oldRest[minIdx];

assert forall i, j :: 0 <= i < j < |oldSorted| ==> oldSorted[i] <= oldSorted[j];
assert forall i, j :: 0 <= i < |oldSorted| && 0 <= j < |oldRest| ==> oldSorted[i] <=
  ↪ oldRest[j];
assert multiset(oldSorted) + multiset(oldRest) == multiset(inputs);

forall t | 0 <= t < |oldRest|
  ensures m <= oldRest[t]
{
  assert oldRest[t] == rest[t];
  assert m == rest[minIdx];
  assert rest[minIdx] <= rest[t];
}

assert oldRest[minIdx..minIdx + 1] == [m];

```

```

assert oldRest[minIdx..] == [m] + oldRest[minIdx + 1..];
assert oldRest == oldRest[..minIdx] + oldRest[minIdx..];
assert oldRest == oldRest[..minIdx] + [m] + oldRest[minIdx + 1..];

sorted := oldSorted + [m];
rest := oldRest[..minIdx] + oldRest[minIdx + 1..];

assert |sorted| == |oldSorted| + 1;
assert |rest| == |oldRest| - 1;
assert |rest| < |oldRest|;

forall i, j | 0 <= i < j < |sorted|
  ensures sorted[i] <= sorted[j]
{
  if j < |oldSorted| {
    assert sorted[i] == oldSorted[i];
    assert sorted[j] == oldSorted[j];
    assert oldSorted[i] <= oldSorted[j];
  } else {
    assert j == |oldSorted|;
    assert i < |oldSorted|;
    assert sorted[i] == oldSorted[i];
    assert sorted[j] == m;
    assert oldSorted[i] <= oldRest[minIdx];
  }
}

forall i, j | 0 <= i < |sorted| && 0 <= j < |rest|
  ensures sorted[i] <= rest[j]
{
  if j < minIdx {
    assert rest[j] == oldRest[j];
    if i < |oldSorted| {
      assert sorted[i] == oldSorted[i];
      assert oldSorted[i] <= oldRest[j];
    } else {
      assert i == |oldSorted|;
      assert sorted[i] == m;
      assert m <= oldRest[j];
    }
  } else {
    assert minIdx <= j;
    assert j + 1 < |oldRest|;
    assert rest[j] == oldRest[j + 1];
    if i < |oldSorted| {
      assert sorted[i] == oldSorted[i];
      assert oldSorted[i] <= oldRest[j + 1];
    } else {
      assert i == |oldSorted|;
      assert sorted[i] == m;
      assert m <= oldRest[j + 1];
    }
  }
}

```

```

    }
  }
}

assert multiset(oldRest) == multiset(oldRest[..minIdx]) + multiset([m]) + multiset(
  ↪ oldRest[minIdx + 1..]);
assert multiset(sorted) == multiset(oldSorted) + multiset([m]);
assert multiset(rest) == multiset(oldRest[..minIdx]) + multiset(oldRest[minIdx + 1..])
  ↪ ;

calc {
  multiset(sorted) + multiset(rest);
==
  (multiset(oldSorted) + multiset([m])) + (multiset(oldRest[..minIdx]) + multiset(
    ↪ oldRest[minIdx + 1..]));
==
  multiset(oldSorted) + multiset(oldRest);
==
  multiset(inputs);
}
}

assert rest == [];
assert multiset(rest) == multiset([]);
assert multiset(sorted) + multiset(rest) == multiset(sorted);
assert multiset(sorted) == multiset(inputs);
assert multiset(inputs) == multiset(sorted);
}

```

Final Program

```

method p_14_2_selection_sort_interval_seq(inputs: seq<int>, bound: nat) returns (sorted:
  ↪ seq<int>)
  requires forall i :: 0 <= i < |inputs| ==> 0 <= inputs[i] <= bound
  ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
  ensures multiset(inputs) == multiset(sorted)
{
  var rest := inputs;
  sorted := [];

  while |rest| > 0
    invariant multiset(sorted) + multiset(rest) == multiset(inputs)
    invariant forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
    invariant forall i, j :: 0 <= i < |sorted| && 0 <= j < |rest| ==> sorted[i] <= rest[j]
    decreases |rest|
  {
    var minIdx := 0;
    var k := 1;

    while k < |rest|

```

```

invariant 0 <= minIdx < |rest|
invariant 1 <= k <= |rest|
invariant forall t :: 0 <= t < k ==> rest[minIdx] <= rest[t]
decreases |rest| - k
{
  if rest[k] < rest[minIdx] {
    var oldMinIdx := minIdx;
    minIdx := k;
    forall t | 0 <= t < k
      ensures rest[minIdx] <= rest[t]
    {
      assert rest[minIdx] == rest[k];
      assert rest[k] < rest[oldMinIdx];
      assert rest[oldMinIdx] <= rest[t];
    }
    assert rest[minIdx] <= rest[k];
  } else {
    assert rest[minIdx] <= rest[k];
  }

  forall t | 0 <= t < k + 1
    ensures rest[minIdx] <= rest[t]
  {
    if t < k {
      assert rest[minIdx] <= rest[t];
    } else {
      assert t == k;
      assert rest[minIdx] <= rest[k];
    }
  }
  k := k + 1;
}

assert k == |rest|;
assert forall t :: 0 <= t < |rest| ==> rest[minIdx] <= rest[t];

var oldSorted := sorted;
var oldRest := rest;
var m := oldRest[minIdx];

assert forall i, j :: 0 <= i < j < |oldSorted| ==> oldSorted[i] <= oldSorted[j];
assert forall i, j :: 0 <= i < |oldSorted| && 0 <= j < |oldRest| ==> oldSorted[i] <=
  ↪ oldRest[j];
assert multiset(oldSorted) + multiset(oldRest) == multiset(inputs);

forall t | 0 <= t < |oldRest|
  ensures m <= oldRest[t]
{
  assert oldRest[t] == rest[t];
  assert m == rest[minIdx];
  assert rest[minIdx] <= rest[t];
}

```

```

}

assert oldRest[minIdx..minIdx + 1] == [m];
assert oldRest[minIdx..] == [m] + oldRest[minIdx + 1..];
assert oldRest == oldRest[..minIdx] + oldRest[minIdx..];
assert oldRest == oldRest[..minIdx] + [m] + oldRest[minIdx + 1..];

sorted := oldSorted + [m];
rest := oldRest[..minIdx] + oldRest[minIdx + 1..];

assert |sorted| == |oldSorted| + 1;
assert |rest| == |oldRest| - 1;
assert |rest| < |oldRest|;

forall i, j | 0 <= i < j < |sorted|
  ensures sorted[i] <= sorted[j]
{
  if j < |oldSorted| {
    assert sorted[i] == oldSorted[i];
    assert sorted[j] == oldSorted[j];
    assert oldSorted[i] <= oldSorted[j];
  } else {
    assert j == |oldSorted|;
    assert i < |oldSorted|;
    assert sorted[i] == oldSorted[i];
    assert sorted[j] == m;
    assert oldSorted[i] <= oldRest[minIdx];
  }
}

forall i, j | 0 <= i < |sorted| && 0 <= j < |rest|
  ensures sorted[i] <= rest[j]
{
  if j < minIdx {
    assert rest[j] == oldRest[j];
    if i < |oldSorted| {
      assert sorted[i] == oldSorted[i];
      assert oldSorted[i] <= oldRest[j];
    } else {
      assert i == |oldSorted|;
      assert sorted[i] == m;
      assert m <= oldRest[j];
    }
  } else {
    assert minIdx <= j;
    assert j + 1 < |oldRest|;
    assert rest[j] == oldRest[j + 1];
    if i < |oldSorted| {
      assert sorted[i] == oldSorted[i];
      assert oldSorted[i] <= oldRest[j + 1];
    } else {

```

```

        assert i == |oldSorted|;
        assert sorted[i] == m;
        assert m <= oldRest[j + 1];
    }
}

assert multiset(oldRest) == multiset(oldRest[..minIdx]) + multiset([m]) + multiset(
    ↪ oldRest[minIdx + 1..]);
assert multiset(sorted) == multiset(oldSorted) + multiset([m]);
assert multiset(rest) == multiset(oldRest[..minIdx]) + multiset(oldRest[minIdx + 1..])
    ↪ ;

calc {
    multiset(sorted) + multiset(rest);
==
    (multiset(oldSorted) + multiset([m])) + (multiset(oldRest[..minIdx]) + multiset(
        ↪ oldRest[minIdx + 1..]));
==
    multiset(oldSorted) + multiset(oldRest);
==
    multiset(inputs);
}

assert rest == [];
assert multiset(rest) == multiset([]);
assert multiset(sorted) + multiset(rest) == multiset(sorted);
assert multiset(sorted) == multiset(inputs);
assert multiset(inputs) == multiset(sorted);
}

```

Total Token Usage

Input tokens: 380

Output tokens: 9334

Reasoning tokens: 7796

Sum of 'total tokens': 9714

Experiment Timings

Overall Experiment started at 1780648501481, ended at 1780648675690, lasting 174209ms (174.21 seconds)

Iteration #1 started at 1780648501485, ended at 1780648675690, lasting 174205ms (174.21 seconds)